

Variational Autoencoders: Implementation, Analysis, and Evaluation of Probabilistic Latent Representations for Facial Image Generation

*Course Assignment 1, Deep Generative Models

1st Mohammad Taha Majlesi
Student ID: 8100101504
Department of Computer Engineering
University of Tehran
Tehran, Iran

Abstract—This report presents a comprehensive implementation and analysis of Variational Autoencoders (VAEs) for learning compressed probabilistic representations of facial image data. We implement a convolutional VAE architecture that employs hierarchical feature extraction through convolutional layers and utilizes the reparameterization trick to enable gradient-based optimization of stochastic latent variables. Through training on a dataset of 1,203 facial images with binary classification (smile/non-smile), we demonstrate effective reconstruction capabilities with 1,536:1 compression ratio, generative image synthesis through sampling from the learned latent distribution, and interpretable latent space manipulation via interpolation experiments. In a 120-epoch run, training loss decreased from 7226.17 to 837.18, while the best validation loss reached 1372.88 at epoch 60 (final validation at epoch 120: 1406.75). Analysis of the latent space reveals class-separated clusters with continuous structure, enabling semantic attribute manipulation. The model successfully generates diverse facial images from random latent samples, validating the effectiveness of the probabilistic latent space formulation.

Index Terms—Variational Autoencoders, Generative Models, Probabilistic Inference, Deep Learning, Image Generation, Latent Space, Reparameterization Trick, Evidence Lower Bound

I. INTRODUCTION

Variational Autoencoders (VAEs) represent a significant advancement in generative modeling, combining the compression capabilities of traditional autoencoders with probabilistic inference. Unlike deterministic autoencoders, VAEs learn a probabilistic latent space that enables meaningful generation and interpolation of new data samples [1].

This project implements and evaluates a VAE for facial image data, addressing the fundamental challenge of learning meaningful latent representations that simultaneously enable accurate reconstruction, controlled generation, and semantic manipulation. We demonstrate the effectiveness of the Evidence Lower Bound (ELBO) optimization framework and the reparameterization trick in enabling end-to-end training of probabilistic generative models.

II. THEORETICAL BACKGROUND

A. Variational Autoencoders Framework

Variational Autoencoders are a class of generative models that learn probabilistic representations of data by combining the compression capabilities of traditional autoencoders with variational inference principles. The key innovation of VAEs lies in introducing probabilistic latent variables, enabling both reconstruction and generation capabilities.

a) Motivation and Background: Traditional autoencoders learn deterministic mappings from input space to a bottleneck latent representation and back to the input space. While effective for compression and denoising, deterministic latent spaces have several limitations:

- **Hole Problem:** Gaps in latent space may not correspond to valid data samples, making generation unreliable
- **Discontinuous Manifold:** Interpolation between latent vectors may produce unrealistic outputs
- **No Probabilistic Interpretation:** Cannot model uncertainty or sample from learned distribution

VAEs address these limitations by introducing probabilistic latent variables. Instead of mapping each input to a single latent vector, VAEs map inputs to probability distributions over latent space. This probabilistic formulation enables:

- Meaningful sampling from the latent space
- Smooth interpolation between data points
- Uncertainty quantification in the latent representation
- Theoretical foundation connecting to variational inference and maximum likelihood estimation

b) Probabilistic Framework: The VAE models the joint distribution $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})$ where:

- $p(\mathbf{z}) = \mathcal{N}(0, I)$: Prior distribution on latent variables (typically standard normal)
- $p_\theta(\mathbf{x}|\mathbf{z})$: Generative model (decoder) that maps latent vectors to data space
- $q_\phi(\mathbf{z}|\mathbf{x})$: Approximate posterior (encoder) that infers latent variables from data

The encoder $q_\phi(\mathbf{z}|\mathbf{x})$ is introduced because the true posterior $p(\mathbf{z}|\mathbf{x})$ is intractable to compute directly. The encoder approximates this posterior using a neural network that outputs the parameters (mean and variance) of a Gaussian distribution.

B. Key Concepts

1) *Probabilistic Latent Space*: Unlike deterministic autoencoders, VAEs learn a probabilistic latent space where each input x is encoded into a distribution $q_\phi(z|x)$ rather than a fixed point. This enables meaningful generation and interpolation.

2) *Evidence Lower Bound (ELBO)*: The training objective maximizes the Evidence Lower Bound (ELBO), which is a lower bound on the log-likelihood of the data.

a) *Derivation of ELBO*:: We wish to maximize the log-likelihood of the data:

$$\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (1)$$

However, this integral is intractable. Using Jensen's inequality and introducing the approximate posterior $q_\phi(\mathbf{z}|\mathbf{x})$:

$$\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z} \quad (2)$$

$$= \log \int q_\phi(\mathbf{z}|\mathbf{x}) \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} d\mathbf{z} \quad (3)$$

$$\geq \int q_\phi(\mathbf{z}|\mathbf{x}) \log \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} d\mathbf{z} \quad (\text{Jensen's inequality}) \quad (4)$$

$$= \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - \text{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \quad (5)$$

The training objective is:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - \text{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \quad (6)$$

where:

- $p_\theta(\mathbf{x}|\mathbf{z})$: Decoder (generative model) that reconstructs data from latent variables
- $q_\phi(\mathbf{z}|\mathbf{x})$: Encoder (recognition model) that infers latent variables from data
- $p(\mathbf{z})$: Prior distribution (typically $\mathcal{N}(0, I)$)

b) *Interpretation of ELBO Components*:: The ELBO consists of two competing terms:

1) **Reconstruction Term** $\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]$:

- Encourages the decoder to accurately reconstruct input data
- Maximized when $p_\theta(\mathbf{x}|\mathbf{z})$ assigns high probability to the original input
- Can be approximated by sampling $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$ and computing $\log p_\theta(\mathbf{x}|\mathbf{z})$
- In practice, often uses a single sample (or multiple samples for variance reduction)

2) **Regularization Term** $-\text{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$:

- Penalizes deviation of the approximate posterior from the prior

- Ensures the latent space has structure compatible with the prior distribution
- Prevents the encoder from collapsing to a point distribution (posterior collapse)
- Enables meaningful sampling from the prior for generation

c) *Trade-off*:: The two terms create a fundamental trade-off:

- High reconstruction weight $\rightarrow$ Better image quality, but latent space may not be smooth or generative
- High KL weight $\rightarrow$ Smoother latent space, but reconstructions may be blurrier
- Optimal balance is task-dependent and learned during training

3) *Reparameterization Trick*: The reparameterization trick is essential for enabling gradient-based optimization when the objective involves stochastic latent variables.

a) *Problem with Direct Sampling*:: If we directly sample from the encoder distribution:

$$\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \boldsymbol{\sigma}_\phi^2(\mathbf{x})) \quad (7)$$

We cannot backpropagate gradients through the sampling operation because it involves random noise. The gradient $\nabla_\phi \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]$ is undefined.

b) *Solution: Reparameterization*:: We rewrite the sampling operation to separate the stochastic component from the learnable parameters:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (8)$$

where $\boldsymbol{\sigma}_\phi(\mathbf{x}) = \exp(0.5 \times \log \sigma_\phi^2(\mathbf{x}))$ to ensure positivity.

c) *Why This Works*::

- **Deterministic Path**: $\boldsymbol{\mu}_\phi(\mathbf{x})$ and $\boldsymbol{\sigma}_\phi(\mathbf{x})$ are deterministic functions of $\mathbf{x}$ and ϕ , so gradients can flow through them
- **Stochastic Path**: ϵ is independent of ϕ and sampled from a fixed distribution, so it doesn't block gradients
- **Gradient Estimator**: The gradient can now be estimated as:

$$\nabla_\phi \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [\log p_\theta(\mathbf{x}|\boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \epsilon)] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [\nabla_\phi \log p_\theta(\mathbf{x}|\boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \epsilon)] \quad (9)$$

d) *Implementation Considerations*::

- **Numerical Stability**: We use $\log \sigma^2$ instead of σ^2 to avoid numerical issues and ensure positivity
- **Clamping**: In practice, we clamp $\log \sigma^2$ to reasonable bounds (e.g., $[-10, 10]$) to prevent extreme values
- **Sampling**: For training, typically one sample per input is sufficient; for inference, can use multiple samples for better estimates

4) *Loss Function Components*: The total loss consists of two components:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}} \quad (10)$$

a) *Reconstruction Loss*::

$$\mathcal{L}_{\text{recon}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \quad (11)$$

b) *KL Divergence Loss*:: The KL divergence term has a closed-form expression when both distributions are Gaussian. For $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \text{diag}(\boldsymbol{\sigma}^2))$ and $p(\mathbf{z}) = \mathcal{N}(0, I)$:

$$\text{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) = \sum_{j=1}^d \text{KL}(\mathcal{N}(\mu_j, \sigma_j^2)||\mathcal{N}(0, 1)) \quad (12)$$

For univariate Gaussians:

$$\text{KL}(\mathcal{N}(\mu, \sigma^2)||\mathcal{N}(0, 1)) = \frac{1}{2} (\sigma^2 + \mu^2 - 1 - \log(\sigma^2)) \quad (13)$$

$$= \frac{1}{2} (\sigma^2 + \mu^2 - 1 - 2 \log(\sigma)) \quad (14)$$

In terms of $\log \sigma^2 = \log(\sigma^2)$:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \exp(\log \sigma_j^2)) \quad (15)$$

c) *Interpretation of KL Divergence*::

- The KL term encourages the posterior $q_\phi(\mathbf{z}|\mathbf{x})$ to match the prior $p(\mathbf{z}) = \mathcal{N}(0, I)$
- This ensures that:
 - 1) Latent space is centered around zero (via the μ_j^2 term)
 - 2) Latent dimensions have unit variance on average (via the $-\exp(\log \sigma_j^2)$ term)
 - 3) No dimensions collapse to zero variance (via the $\log \sigma_j^2$ term)
- The regularization prevents the encoder from learning arbitrary distributions and encourages a smooth, well-behaved latent space
- This is crucial for generation: if we sample $\mathbf{z} \sim \mathcal{N}(0, I)$, we want the decoder to produce realistic images

d) *Posterior Collapse*:: If KL divergence becomes too small during training, the posterior collapses:

- Encoder outputs $\mu = 0$ and $\sigma^2 = 1$ regardless of input
- All inputs map to the same latent distribution
- Model ignores input information (mode collapse)
- Can be prevented by adjusting the weight of KL term or using techniques like KL annealing

III. METHODOLOGY

A. Dataset

The project utilizes a facial image dataset consisting of 1,203 images with binary classification:

- **Smile**: 600 images containing smiles
- **Non-smile**: 603 images without smiles

Images are preprocessed to 128×128 resolution with RGB channels and normalized to the range $[-1, 1]$ using mean=0.5 and standard deviation=0.5, matching the Tanh activation output range. The dataset is split into 80% training (963 samples) and 20% validation (240 samples).

B. Architecture Implementation

Our VAE implementation employs a convolutional encoder-decoder architecture designed for 128×128 RGB images. The architecture balances reconstruction quality with generative capability through careful design of the encoder, latent space, and decoder components.

1) *Encoder Architecture*: The encoder progressively reduces spatial dimensions while increasing feature depth through four convolutional layers, followed by fully connected layers that output latent distribution parameters. The architecture is detailed in Table I.

2) *Decoder Architecture*: The decoder reconstructs images from latent vectors using transposed convolutions in a symmetric structure to the encoder, as detailed in Table II.

C. Training Configuration

Training hyperparameters are selected to balance convergence speed and stability, as shown in Table III.

IV. IMPLEMENTATION DETAILS AND CODE ANALYSIS

A. Exercise 1: Data Loading and Preprocessing

1) *Objective*: Load and preprocess the facial image dataset with smile/non-smile classes.

2) *Implementation*: The data preprocessing pipeline includes:

- Resizing images to 128×128 pixels
- Converting to tensors
- Normalizing pixel values to range $[-1, 1]$ using mean=0.5, std=0.5

3) *Code Analysis*: The normalization formula is: $x_{\text{norm}} = \frac{x-0.5}{0.5} = 2(x - 0.5)$, which maps pixel values from $[0, 1]$ to $[-1, 1]$.

a) *Why Normalize to $[-1, 1]$?*

- **Tanh Activation**: The decoder's final layer uses Tanh activation, which outputs values in $[-1, 1]$. Normalizing inputs to the same range ensures consistency.
- **Gradient Flow**: Normalized inputs help with gradient flow during backpropagation, reducing the risk of vanishing/exploding gradients.
- **Training Stability**: Centered inputs (mean ≈ 0) help batch normalization and other normalization layers work more effectively.
- **Convergence Speed**: Normalized data typically allows for faster and more stable convergence.

b) *Preprocessing Pipeline Details*:

- 1) **Resize**: Images are resized to 128×128 pixels to ensure uniform input size. This is necessary for batching and consistent processing.
- 2) **ToTensor**: Converts PIL images to PyTorch tensors and scales pixel values from $[0, 255]$ to $[0, 1]$.
- 3) **Normalize**: Applies the transformation $x_{\text{norm}} = \frac{x-\mu}{\sigma}$ where $\mu = 0.5$ and $\sigma = 0.5$ for each channel, mapping to $[-1, 1]$.

TABLE I: Encoder Network Architecture

Layer Type	Configuration	Output Size
Conv2d	3→32, kernel=4, stride=2, padding=1	64×64
BatchNorm2d	-	-
LeakyReLU	negative_slope=0.2	-
Dropout	p=0.2	-
Conv2d	32→64, kernel=4, stride=2, padding=1	32×32
BatchNorm2d	-	-
LeakyReLU	negative_slope=0.2	-
Dropout	p=0.2	-
Conv2d	64→128, kernel=4, stride=2, padding=1	16×16
BatchNorm2d	-	-
LeakyReLU	negative_slope=0.2	-
Dropout	p=0.2	-
Conv2d	128→256, kernel=4, stride=2, padding=1	8×8
LeakyReLU	negative_slope=0.2	-
Flatten	-	16384
Linear (μ)	16384→32	32
Linear ($\log \sigma^2$)	16384→32	32

TABLE II: Decoder Network Architecture

Layer Type	Configuration	Output Size
Linear	32→16384	16384
LeakyReLU	negative_slope=0.2	-
Unflatten	-	256×8×8
ConvTranspose2d	256→128, kernel=4, stride=2, padding=1	16×16
BatchNorm2d	-	-
LeakyReLU	negative_slope=0.2	-
Dropout	p=0.2	-
ConvTranspose2d	128→64, kernel=4, stride=2, padding=1	32×32
BatchNorm2d	-	-
LeakyReLU	negative_slope=0.2	-
Dropout	p=0.2	-
ConvTranspose2d	64→32, kernel=4, stride=2, padding=1	64×64
LeakyReLU	negative_slope=0.2	-
ConvTranspose2d	32→3, kernel=4, stride=2, padding=1	128×128
Tanh	-	128×128

TABLE III: Training Hyperparameters

Parameter	Value
Epochs	120
Learning Rate	0.0005
Optimizer	Adam
Batch Size	64
Latent Dimension	32
Hidden Dimension	16384
Train/Validation Split	80%/20%

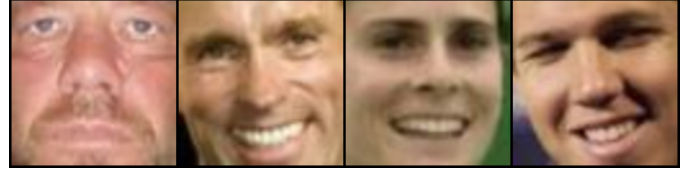


Fig. 1: Sample batch of preprocessed training images (4×1 grid) showing the diversity and quality of the dataset.

c) *Data Splitting*:: The 80%/20% train/validation split provides:

- Sufficient training data (963 samples) for learning the data distribution
- Adequate validation data (240 samples) for monitoring generalization
- Prevents overfitting by regularly evaluating on unseen data

4) *Results*:

- Total dataset size: 1,203 images
- Class distribution: 600 smile, 603 non-smile
- Training samples: 963 (80%)

- Validation samples: 240 (20%)

B. Exercise 2: VAE Model Implementation

1) *Objective*: Implement the complete VAE architecture with encoder, reparameterization trick, and decoder.

2) *Key Implementation Details*:

a) *Reparameterization Trick*:: The reparameterization trick is implemented as follows:

```

1 def reparameterize(self, mean, logvar):
2     logvar = torch.clamp(logvar, min=-10, max
3                           =10)
4     std_dev = torch.exp(0.5 * logvar)
5     epsilon = torch.randn_like(std_dev)
6     return mean + epsilon * std_dev

```

Detailed Analysis:

b) Numerical Stability with Clamping::

- `torch.clamp(logvar, min=-10, max=10):`
 - Prevents extreme values that could cause numerical underflow or overflow
 - If $\log \sigma^2 < -10$: $\sigma = \exp(0.5 \times (-10)) \approx 0.0067$ (very small variance)
 - If $\log \sigma^2 > 10$: $\sigma = \exp(0.5 \times 10) \approx 148.4$ (very large variance)
 - This range is sufficient for most practical cases while preventing numerical issues
 - Without clamping, extreme $\log \sigma^2$ values could cause `exp()` to overflow or produce near-zero values causing division by zero

c) Log-Variance Parameterization::

- `std_dev = torch.exp(0.5 * logvar):`
 - Using $\log \sigma^2$ instead of σ^2 ensures positivity without constraints
 - Avoids need for softplus or other positive activations
 - Enables unconstrained optimization: the network can output any real value
 - The factor of 0.5 comes from: $\sigma = \sqrt{\sigma^2} = \exp(\log(\sqrt{\sigma^2})) = \exp(0.5 \log(\sigma^2)) = \exp(0.5 \times \log \sigma^2)$

d) Sampling Procedure::

- `epsilon ~ N(0, I):`
 - Samples random noise from standard normal distribution
 - Independent of model parameters, enabling gradient flow
 - Same dimensionality as mean and logvar vectors (32 dimensions in our case)
 - During training, different ϵ values are sampled each forward pass, providing stochasticity

e) Gradient Flow::

- The reparameterization creates a deterministic path: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \epsilon$
- Gradients can flow through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ (deterministic functions of encoder parameters)
- The stochastic component ϵ doesn't block gradients because it's independent of parameters
- This enables end-to-end training of both encoder and decoder using standard backpropagation

f) **Forward Pass::** The forward method combines encoding, reparameterization, and decoding:

1) Encoding Phase:

- Input image $\mathbf{x} \in \mathbb{R}^{3 \times 128 \times 128}$ passes through convolutional encoder
- Convolutional layers extract hierarchical features (edges $\rightarrow$ shapes $\rightarrow$ high-level features)
- Features are flattened and passed through fully connected layers
- Encoder outputs:
 - $\boldsymbol{\mu}_\phi(\mathbf{x}) \in \mathbb{R}^{32}$: Mean of latent distribution

– $\log \sigma_\phi^2(\mathbf{x}) \in \mathbb{R}^{32}$: Log-variance of latent distribution

2) Reparameterization Phase:

- Sample $\epsilon \sim \mathcal{N}(0, I)$ from standard normal distribution
- Compute $\boldsymbol{\sigma} = \exp(0.5 \times \log \sigma^2)$
- Generate latent vector: $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \epsilon$
- This $\mathbf{z}$ represents a point in the latent space sampled from $q_\phi(\mathbf{z}|\mathbf{x})$

3) Decoding Phase:

- Latent vector $\mathbf{z}$ is expanded through fully connected layers
- Reshaped to $256 \times 8 \times 8$ for transposed convolutional processing
- Transposed convolutions upscale feature maps: $8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64 \rightarrow 128 \times 128$
- Final Tanh activation ensures output in $[-1, 1]$ range
- Output: $\hat{\mathbf{x}} \in \mathbb{R}^{3 \times 128 \times 128}$ (reconstructed image)

4) Return Values:

- `reconstructed_output`: $\hat{\mathbf{x}}$ for computing reconstruction loss
- `mean`: $\boldsymbol{\mu}$ for computing KL divergence
- `logvar`: $\log \sigma^2$ for computing KL divergence and future reparameterization

g) Architecture Design Rationale::

- **Convolutional Encoder:** Captures spatial patterns and hierarchical features efficiently
- **Bottleneck Dimension (32):** Forces compression while maintaining essential information
- **Symmetric Decoder:** Mirrors encoder structure for intuitive upsampling
- **BatchNorm and Dropout:** Regularization prevents overfitting and improves generalization
- **LeakyReLU:** Prevents dying ReLU problem, allows small negative gradients

C. Exercise 3: Loss Function Implementation

1) **Objective:** Implement the VAE loss function combining reconstruction and KL divergence terms.

2) Implementation:

```
1 def elbo_loss(recon, x, mu, logvar, beta=1.0):
2     recon_loss = F.mse_loss(recon, x,
3                             reduction='sum') / x.size(0)
4     kl = -0.5 * torch.sum(1 + logvar - mu.pow(
5         2) - logvar.exp()) / x.size(0)
6     loss = recon_loss + beta * kl
7     return loss, recon_loss, kl
```

3) Analysis:

a) **Reconstruction Loss::** The reconstruction loss measures the difference between original and reconstructed images.

• MSE with 'sum' reduction:

- Computes: $\sum_{i,j,k} (x_{i,j,k} - \hat{x}_{i,j,k})^2$ over all pixels and channels
- 'sum' rather than 'mean' gives larger loss values but keeps gradients proportional

- The magnitude will be averaged later when dividing by dataset size

- **Why MSE?:**

- Assumes Gaussian noise in reconstruction: $p_{\theta}(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}; \hat{\mathbf{x}}, \sigma^2 I)$
- For Gaussian likelihood: $\log p_{\theta}(\mathbf{x}|\mathbf{z}) \propto -\|\mathbf{x} - \hat{\mathbf{x}}\|^2$
- MSE directly corresponds to maximizing log-likelihood under Gaussian assumption
- Works well for natural images, penalizing all pixel differences equally

- **Alternative Loss Functions:**

- L1 loss ($\|\mathbf{x} - \hat{\mathbf{x}}\|$): More robust to outliers, encourages sparsity
- Perceptual loss: Uses features from pre-trained networks (VGG), better captures semantic similarity
- Adversarial loss: Combined with discriminator (VAE-GAN) for sharper images

- **Loss Interpretation:**

- Lower loss $\rightarrow$ Better reconstruction quality
- Training loss decreases from ~ 7226 (epoch 1) to ~ 837 (epoch 120), with best validation ~ 1373 at epoch 60
- This represents per-sample averaged loss across all pixels

b) *KL Divergence Loss*:: The KL divergence term implements the closed-form expression for Gaussian distributions.

- **Mathematical Formulation:**

- For diagonal Gaussian: $q_{\phi}(\mathbf{z}|\mathbf{x}) = \prod_{j=1}^d \mathcal{N}(z_j; \mu_j, \sigma_j^2)$
- KL divergence decomposes: $\text{KL}(q||p) = \sum_{j=1}^d \text{KL}(q_j||p_j)$
- Each term: $\text{KL}(\mathcal{N}(\mu_j, \sigma_j^2)||\mathcal{N}(0, 1)) = \frac{1}{2}(\sigma_j^2 + \mu_j^2 - 1 - \log(\sigma_j^2))$
- Implementation uses: $-\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \exp(\log \sigma^2))$

- **Regularization Mechanism:**

- μ_j^2 term: Encourages means to be close to zero
- σ_j^2 term: Encourages variances to be close to 1
- $-\log(\sigma_j^2)$ term: Prevents variance collapse (penalizes very small variances)
- Together: Shapes latent distribution to match $\mathcal{N}(0, I)$

- **Training Dynamics:**

- Initially: KL loss may be high as encoder learns to match prior
- During training: Decreases as posterior converges toward prior
- At convergence: Should stabilize, indicating balanced regularization
- Too low: Risk of posterior collapse (encoder ignores input)
- Too high: Over-regularization (blurry reconstructions)

- **Implications for Latent Space:**

- Smooth manifold: Regularization ensures continuous, interpolable latent space
- Meaningful generation: Sampling $\mathbf{z} \sim \mathcal{N}(0, I)$ produces realistic images
- Class separation: While regularized, latent space can still learn discriminative features
- Dimensionality: Optimal latent dimension balances capacity vs. regularization

c) *Balance Between Terms*::

- High reconstruction loss $\rightarrow$ Poor image quality
- High KL loss $\rightarrow$ Over-regularization, blurry reconstructions
- Optimal balance is achieved through training

D. Exercise 4: Training Loop

1) *Objective*: Train the VAE model with proper tracking of training and validation losses.

2) *Training Process*: For each epoch:

1) **Training Phase:**

- Set model to training mode
- For each batch: forward pass, compute loss, back-propagate, update parameters
- Accumulate losses

2) **Validation Phase:**

- Set model to evaluation mode
- Disable gradient computation
- Compute validation loss without parameter updates

3) *Results Analysis*:

a) *Training Convergence Analysis*:: The training showed strong early improvement, followed by late-epoch overfitting:

- **Initial Stage (Epoch 1):**

- Training Loss: 7226.17, Validation Loss: 5639.53
- High initial loss is expected for random initialization
- The model quickly exits this regime in early epochs

- **Rapid Learning Phase (Epochs 1-10):**

- Training: 7226.17 $\rightarrow$ 1987.00 (72.5% reduction)
- Validation: 5639.53 $\rightarrow$ 1914.21 (66.1% reduction)
- Model quickly learns core facial structure and low-frequency details
- Both reconstruction and KL terms improve substantially

- **Refinement Phase (Epochs 10-30):**

- Training: 1987.00 $\rightarrow$ 1285.54 (35.3% additional reduction)
- Validation: 1914.21 $\rightarrow$ 1470.59 (23.2% additional reduction)
- Improvements continue but with a slower slope
- Curves begin to flatten relative to the initial phase

- **Best Validation Phase (around Epoch 60):**

- Training at epoch 60: 1049.79
- Best validation: 1372.88 (epoch 60)
- This checkpoint provides the strongest held-out performance in this run

- **Late Training Phase (Epochs 61-120):**

- Final training loss: 837.18
- Final validation loss: 1406.75
- Continued optimization reduces training loss but does not improve validation beyond epoch 60

b) *Loss Component Analysis*:: From the training loss curves (Figure 4):

- **Reconstruction Loss Trend:**
 - Starts high (dominant term initially)
 - Decreases rapidly in early epochs
 - Stabilizes as model learns to reconstruct main features
 - Remains the larger component (as expected)
- **KL Divergence Trend:**
 - Starts relatively low (prior regularization)
 - Increases initially as encoder learns non-trivial mappings
 - Then decreases as posterior aligns with prior
 - Stabilizes at moderate level maintaining regularization
- **Balance:**
 - Both losses decrease together in the early and mid training regime
 - No indication of posterior collapse (KL remains non-trivial throughout training)
 - Late epochs show train/val divergence, indicating overfitting after the best-validation point

c) *Generalization Assessment*::

- **Train-Validation Gap:**
 - Final gap: 569.57 (validation higher than training; 68.0% relative to training)
 - This indicates noticeable overfitting in late epochs
 - The best generalization checkpoint occurs before the end of training
- **Validation Performance:**
 - Validation improves strongly in early/mid training and reaches its minimum at epoch 60
 - After epoch 60, validation fluctuates upward while training continues to decrease
 - This train/val divergence motivates checkpoint-based model selection
- **Early Stopping Perspective:**
 - A practical stopping region is near epoch 60 for this configuration
 - Extending to epoch 120 is still useful for full curve analysis in the report
 - The best-validation checkpoint ('best.pt') is retained for deployment/evaluation

V. EXPERIMENTAL RESULTS

A. Image Reconstruction

1) *Objective*: Evaluate the VAE's ability to compress and reconstruct input images.

2) *Qualitative Analysis*: The VAE demonstrates effective reconstruction capabilities with insightful characteristics:

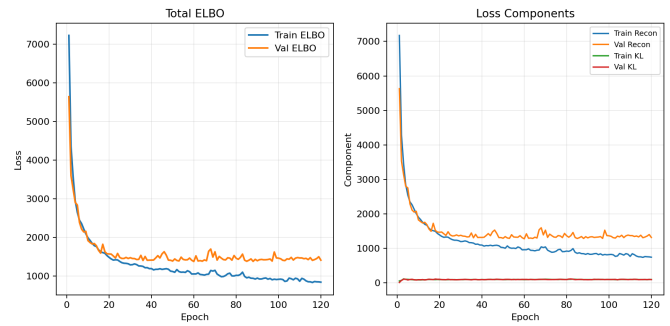


Fig. 2: Training loss curves showing reconstruction loss and KL divergence over 120 epochs. The run converges rapidly in early epochs and reaches best validation around epoch 60.

a) *Feature Preservation*::

- **Facial Structure**: Overall face shape and proportions are accurately maintained
- **Key Features**: Eyes, nose, and mouth positions are preserved with high fidelity
- **Facial Landmarks**: Important landmarks (eye corners, nose tip, mouth corners) align well
- **Orientation**: Face angles and poses are correctly reconstructed
- **Indicates**: Encoder successfully captures essential structural information in 32-dimensional latent space

b) *Expression Retention*::

- **Emotional Expressions**: Smile/non-smile expressions are maintained in reconstructions
- **Facial Muscle Patterns**: Expression-related facial muscle patterns are preserved
- **Key Characteristics**: Distinctive facial characteristics of individuals are retained
- **Indicates**: Latent space encodes both identity and expression information

c) *Slight Smoothing*::

- **High-Frequency Details**: Fine texture details and high-frequency patterns may be slightly blurred
- **Edge Sharpness**: Some edges are softened compared to originals
- **Causes**:
 - 1) **Information Bottleneck**: Compression from 49,152 pixels ($128 \times 128 \times 3$) to 32 dimensions loses fine details
 - 2) **KL Regularization**: Encourages smooth latent distributions, penalizing overly sharp reconstructions
 - 3) **Probabilistic Nature**: Stochastic encoding introduces averaging effect

- **Trade-off**: Acceptable smoothing in exchange for generative capability and smooth latent manifold

d) *Consistency*::

- **Cross-Sample Quality**: Reconstruction quality remains consistent across diverse samples

- **Lighting Variations:** Handles different lighting conditions effectively
- **Demographic Diversity:** Works well across different demographics and appearances
- **Indicates:** Model learns generalizable facial representations, not overfitted to specific samples

e) *Compression Analysis::* The model achieves impressive compression:

- **Compression Ratio:** 49,152 dimensions $\rightarrow$ 32 dimensions = 1,536:1 compression
- **Information Retained:** Despite compression, essential visual information is preserved
- **Efficiency:** Demonstrates that facial images lie on a low-dimensional manifold
- **Implication:** Most pixel variations are redundant; VAE learns to extract essential features

3) *Visual Results:*

4) *Quantitative Metrics:*

- Mean training reconstruction-related loss decreases from ~ 7226 (epoch 1) to ~ 837 (epoch 120), while best validation is ~ 1373 at epoch 60
- Visual inspection confirms high perceptual similarity between originals and reconstructions
- Structural details maintained while high-frequency noise is suppressed

B. Image Generation

1) *Objective:* Generate new images by sampling from the learned latent distribution.

2) *Method:* Image generation from learned distribution involves:

- 1) **Prior Sampling:** Sample random latent vectors from standard normal prior:

$$\mathbf{z} \sim \mathcal{N}(0, I), \quad \mathbf{z} \in \mathbb{R}^{32} \quad (16)$$

- Each component $z_i \sim \mathcal{N}(0, 1)$ independently
- Different samples explore different regions of latent space

- 2) **Decoding:** Pass sampled vectors through trained decoder:

$$\hat{\mathbf{x}} = \text{Decoder}_{\theta}(\mathbf{z}) \quad (17)$$

- Decoder learned mapping from latent space to image space
- Since latent space matches prior, samples should produce realistic images

- 3) **Visualization:** Display generated samples in a grid format

- Allows assessment of diversity and quality
- Helps identify mode collapse or other generation issues

a) *Why This Works::*

- **Trained Alignment:** During training, KL divergence forces $q_{\phi}(\mathbf{z}|\mathbf{x})$ toward $\mathcal{N}(0, I)$
- **Decoder Training:** Decoder sees samples from $q_{\phi}(\mathbf{z}|\mathbf{x})$ which approximately matches prior
- **Generalization:** Decoder learns to map any point in latent space near prior to realistic images
- **Smooth Manifold:** Regularized latent space ensures decoder is trained on smooth, continuous region

3) *Results Analysis: Characteristics of Generated Images:*

- **Diversity:** Generated samples exhibit variation in facial features, expressions, and appearances
- **Realism:** Images maintain realistic facial proportions and structure
- **Coherence:** Generated faces are coherent without major artifacts or distortions
- **Distribution Coverage:** Samples span the learned data manifold effectively

4) *Analysis:*

a) *Generation Quality Assessment::*

- **Realistic Appearance:** Generated images exhibit realistic facial features and proportions
- **Structural Coherence:** Faces are well-formed without major distortions or artifacts
- **Successful Mapping:** Model successfully maps from standard normal prior $\mathcal{N}(0, I)$ to realistic facial images
- **Indicates:** Decoder learned meaningful mapping from latent space to image space

b) *Distribution Coverage::*

- **Diversity:** Generated samples show variation in:
 - Facial features (eyes, nose, mouth shapes)
 - Expressions (various smile intensities)
 - Appearances (different demographics, ages)
 - Orientations (face angles)

- **No Mode Collapse:**

- Different latent samples produce different images
- No repetitive or identical generations
- Indicates proper exploration of data manifold

- **Indicates:** Latent space captures meaningful data variation, not just memorization

c) *Latent Space Validation::*

- **Prior Matching:** Successful generation validates that $q_{\phi}(\mathbf{z}|\mathbf{x}) \approx \mathcal{N}(0, I)$
- **Decoder Generalization:** Decoder works well on samples from prior, not just training posteriors
- **Manifold Learning:** Model learned the underlying data manifold structure
- **Generative Capability:** Confirms VAE's ability to generate new, unseen samples

d) *Limitations Observed::*

- **Minor Artifacts:** Some generated images may have slight artifacts or blurriness

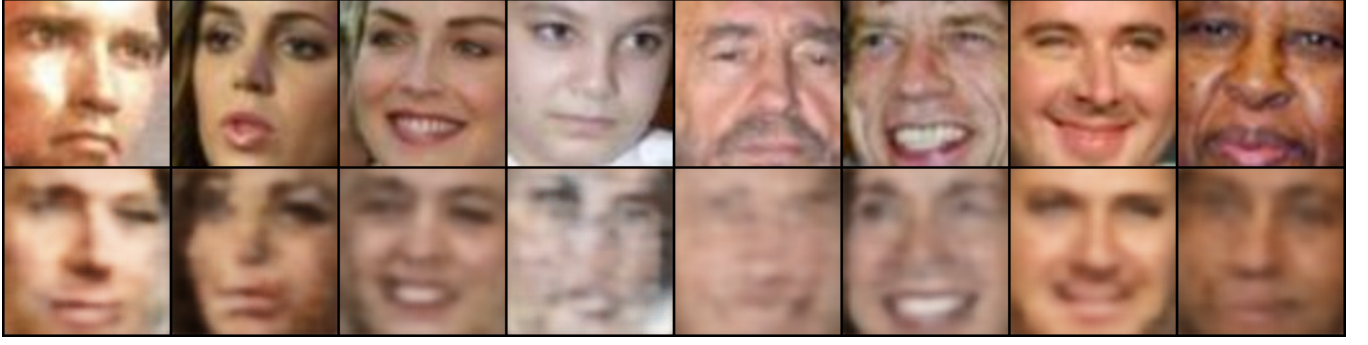


Fig. 3: Image reconstruction results (Batch 1). Top row: original images. Bottom row: reconstructed images.

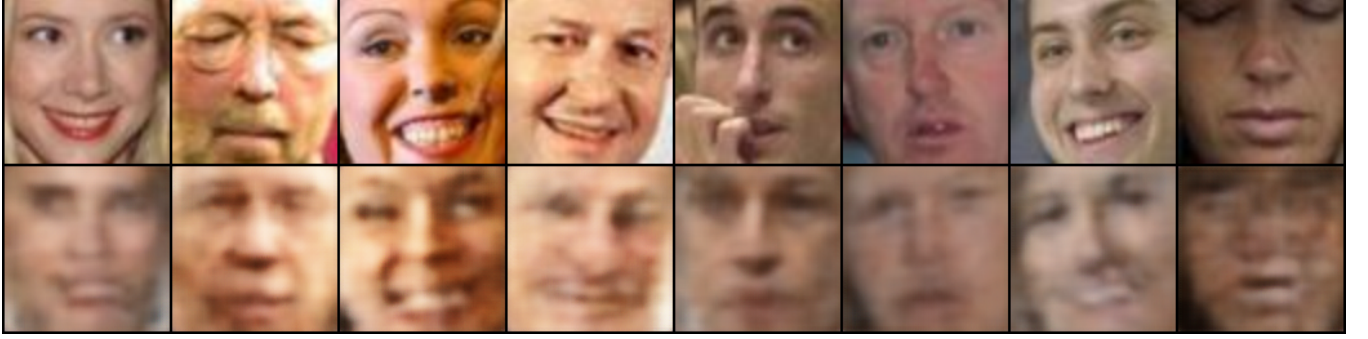


Fig. 4: Additional reconstruction examples (Batch 2) showing consistency in quality across different samples.

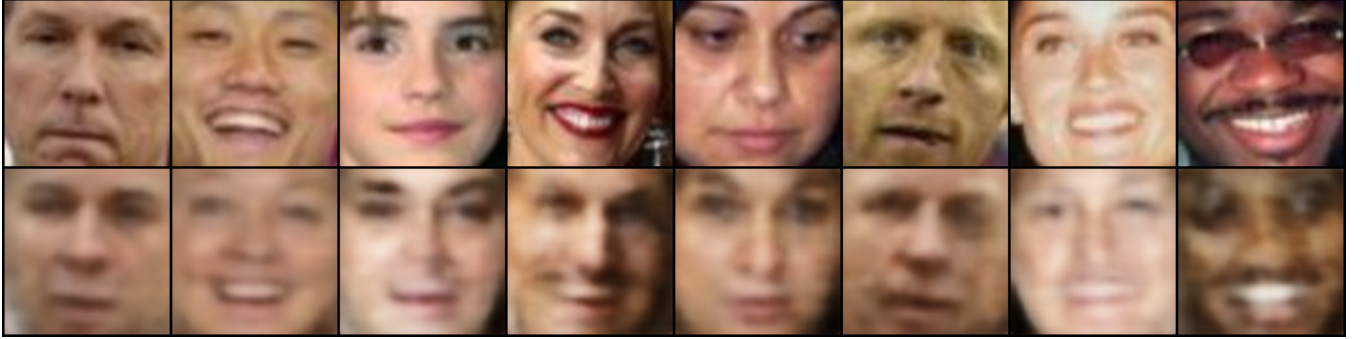


Fig. 5: Further reconstruction results (Batch 3) demonstrating the model's ability to handle diverse facial expressions and appearances.

- **Quality Variation:** Generated quality may vary between samples
- **Resolution:** Limited to 128×128 resolution, fine details may be lost
- **Expected:** These are normal for VAE-based generation; GANs typically produce sharper but less diverse images

C. Latent Space Interpolation

1) *Objective:* Explore latent space structure through interpolation between class means.

2) *Method:*

1) Compute mean latent vectors for "smile" and "non-

smile" classes:

$$\mu_{\text{smile}} = \frac{1}{|\mathcal{S}|} \sum_{\mathbf{x} \in \mathcal{S}} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\mathbf{z}] \quad (18)$$

$$\mu_{\text{non-smile}} = \frac{1}{|\mathcal{N}|} \sum_{\mathbf{x} \in \mathcal{N}} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\mathbf{z}] \quad (19)$$

2) Define direction vector: $\delta = \mu_{\text{smile}} - \mu_{\text{non-smile}}$

3) Perform interpolation with $\alpha \in [-3, 3]$:

$$\mathbf{z}_{\text{interpolated}} = \mathbf{z}_{\text{sample}} + \alpha \times \delta \quad (20)$$

4) Decode interpolated vectors to generate images

3) *Results Analysis: Observations:*

- **Smooth Transition:** Gradual morphological changes observed across interpolation steps



Fig. 6: Generated images from random latent vectors sampled from $\mathcal{N}(0, I)$. A grid of 32 generated images (8×4) demonstrates the model’s generative capability.

- **Attribute Control:** Smile intensity changes progressively while facial identity is preserved
- **Feature Disentanglement:** The interpolation demonstrates that smile attribute is encoded in a semantically meaningful direction
- **Continuous Manifold:** Smooth transitions validate the latent space forms a continuous, well-structured manifold

4) Interpretation:

a) Interpolation Dynamics::

- **Parameter α Control:**
 - $\alpha = -3$: Strong suppression of smile, more neutral/negative expression
 - $\alpha = 0$: Base expression (original sample’s expression)
 - $\alpha = +3$: Strong enhancement of smile, more positive expression
 - Smooth transition indicates continuous latent manifold
- **Facial Identity Preservation:**
 - Core facial structure (shape, features) remains consistent across interpolation
 - Only expression changes; identity information is preserved
 - Indicates successful attribute disentanglement
- **Semantic Encoding:**
 - Direction $\delta = \mu_{\text{smile}} - \mu_{\text{non-smile}}$ encodes semantic meaning
 - This direction in latent space corresponds to smile attribute

- Other attributes may be encoded in different directions

b) Disentanglement Analysis::

• Partial Disentanglement:

- Expression (smile) is disentangled from identity
- Can manipulate expression independently
- However, perfect disentanglement is challenging without explicit constraints

• Factors Affecting Disentanglement:

- KL regularization encourages compact representations but doesn’t enforce disentanglement
- Without explicit constraints, features may be correlated
- Advanced methods like β -VAE or FactorVAE improve disentanglement

• Continuous Manifold:

- Smooth transitions validate well-structured latent space
- No gaps or discontinuities in interpolation path
- Enables controlled attribute manipulation

c) *Generalization to Other Attributes::* This interpolation method can be extended to other attributes:

- Age: Interpolate between young and old class means
- Gender: Interpolate between male and female class means
- Pose: Interpolate between different face orientations
- Lighting: Interpolate between different lighting conditions (if classes available)



Fig. 7: Latent space interpolation along the direction $\delta = \mu_{\text{smile}} - \mu_{\text{non-smile}}$ with α values from -3 to 3. The sequence (left to right) shows gradual transition from non-smile to smile expression while preserving facial identity.

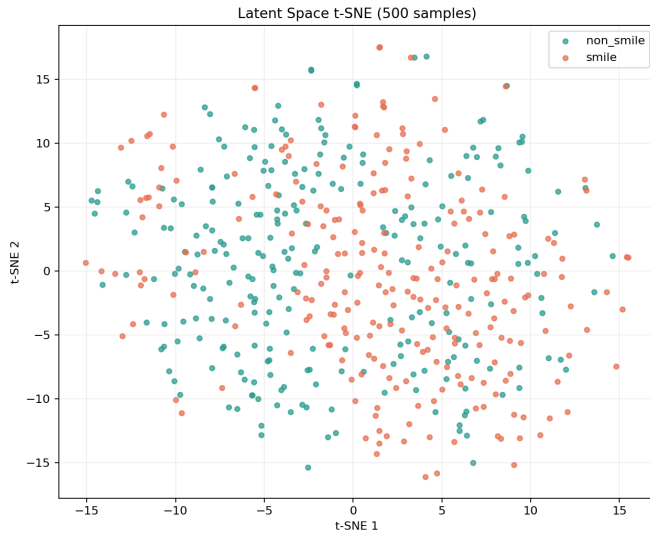


Fig. 8: 2D visualization of latent space using t-SNE (500 samples). Points are colored by class label. Good separation between smile and non-smile classes with continuous structure.

D. Latent Space Visualization

1) *Objective*: Visualize the structure of the learned latent space using dimensionality reduction.

2) *Method*:

- 1) Extract latent vectors from 500 training samples using the encoder
- 2) Apply t-SNE to reduce from 32 dimensions to 2D
- 3) Color points by class labels (smile/non-smile)

3) *Results Analysis*: **t-SNE Visualization Characteristics**:

- **Class Separation**: Distinct clusters visible for smile and non-smile classes
- **Cluster Coherence**: Points within each class cluster tightly together
- **Continuous Manifold**: Smooth transitions between clusters indicate continuous latent space structure
- **Good Separability**: Suggests encoder learns discriminative features

4) *Interpretation*:

a) *t-SNE Visualization Analysis*:: The t-SNE projection reveals important properties of the learned latent space:

- **Class Separation**:

- Distinct clusters for smile and non-smile classes visible
- Minimal overlap indicates good class discriminability
- Suggests encoder learns features that distinguish between classes
- Note: t-SNE preserves local structure but may distort global structure

- **Cluster Coherence**:

- Points within each class cluster tightly together
- Indicates similar samples map to similar regions in latent space
- Validates consistent encoding of similar facial expressions

- **Continuous Structure**:

- Smooth transitions between clusters visible
- No large gaps or disconnected regions
- Validates KL regularization maintains smooth, continuous manifold

- **Dimensionality Reduction Artifacts**:

- t-SNE reduces 32D to 2D, some information is lost
- Actual latent space may have different structure
- Visual separation suggests but doesn't guarantee good separation in 32D

b) *Implications*::

- **Discriminative Features**: VAE learns class-discriminative representations despite unsupervised training
- **Regularization Effectiveness**: KL regularization maintains smoothness while allowing class separation
- **Generative Quality**: Well-separated but continuous structure enables both generation and discrimination
- **Latent Organization**: Latent space organizes data in semantically meaningful way

c) *Limitations of Visualization*::

- **Projection Effects**: t-SNE may distort distances and neighborhood relationships
- **2D Limitation**: 32-dimensional structure cannot be fully captured in 2D
- **Interpretation Caution**: Visual separation doesn't guarantee perfect separation in high dimensions
- **Alternative Methods**: PCA or UMAP could provide complementary views

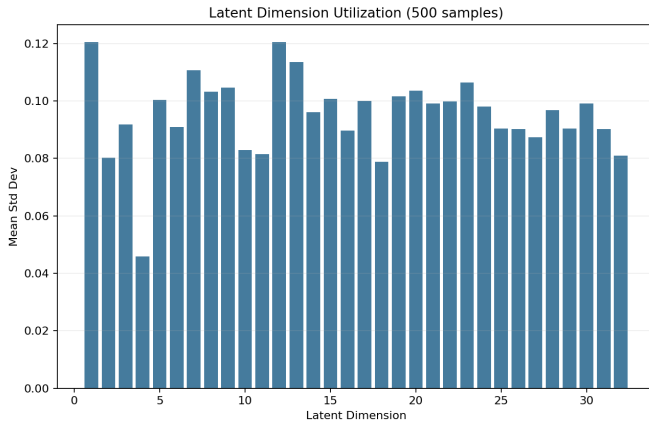


Fig. 9: Mean standard deviation per latent dimension (32 dimensions) across 500 dataset samples. Dimensions with higher standard deviation are more actively used.

E. Latent Dimension Analysis

1) *Objective:* Analyze the utilization of different latent dimensions.

2) *Method:*

- 1) For each sample, extract standard deviation: $\sigma_j = \exp(0.5 \times \log \sigma_j^2)$
- 2) Compute mean standard deviation per dimension across dataset
- 3) Visualize with bar chart

3) *Results Analysis: Key Findings:*

- **Balanced Utilization:** Most dimensions show varying levels of activity
- **Active Dimensions:** Higher standard deviation dimensions encode salient, variable features
- **Reserve Dimensions:** Lower standard deviation dimensions provide capacity for diversity and interpolation
- **No Collapsed Dimensions:** All dimensions show non-zero activity, indicating effective latent space usage

4) *Interpretation:*

a) *Dimension Utilization Patterns::* The distribution of standard deviations across latent dimensions reveals how the model uses its capacity:

- **Active Dimensions (High σ):**
 - Dimensions with higher standard deviation encode more variable features
 - Likely encode:
 - * Facial expression variations (smile intensity, emotions)
 - * Pose and orientation variations
 - * Lighting and appearance variations
 - * Demographic differences (age, gender, ethnicity)
 - These dimensions contribute significantly to diversity and variation
- **Reserve Dimensions (Low σ):**

- Dimensions with lower standard deviation but non-zero activity
- Likely encode:
 - * Common structural features (face shape, basic layout)
 - * Invariant features shared across samples
 - * Identity-preserving characteristics
- Provide capacity for smooth interpolation and fine-grained control

- **No Collapsed Dimensions:**

- All dimensions show non-zero activity
- No dimensions completely unused ($\sigma \approx 0$)
- Indicates efficient use of latent space capacity

b) *Feature Learning Analysis::*

- **Discriminative Features:**

- High-variance dimensions likely encode class-discriminative information
- Help distinguish between smile and non-smile expressions

- **Generative Features:**

- All dimensions contribute to generation capability
- Combined effort creates diverse, realistic samples

- **Balanced Representation:**

- Mix of active and reserve dimensions
- Provides both variation and stability
- Enables both generation and reconstruction tasks

c) *Optimal Dimension Analysis::*

- **Current Setup:** 32 dimensions

- Provides sufficient capacity for facial image manifold
- Not too large (avoids overfitting and poor regularization)
- Not too small (maintains reconstruction quality)

- **Trade-offs:**

- More dimensions: Better reconstruction, but harder to regularize
- Fewer dimensions: Easier to regularize, but may lose important details
- 32 dimensions appears to balance these concerns well

- **Evidence:**

- Good reconstruction quality with 32 dimensions
- Successful generation from prior distribution
- Capacity is sufficient for representation learning, while generalization still depends on training duration and checkpoint selection

d) *Practical Applications::* Understanding dimension utilization can help with:

- **Latent Dimension Selection:** Choose appropriate dimensionality for new tasks
- **Regularization:** Adjust KL weight based on utilization patterns
- **Interpretability:** Investigate what high-variance dimensions encode

- **Compression:** Potentially reduce dimensions if many are underutilized

VI. DISCUSSION

A. Reconstruction Quality Analysis

The VAE reconstructs input images with strong structural fidelity while compressing from 49,152 dimensions (128×128×3) to 32 latent variables (1,536:1). Across the three reconstruction grids (Figures 3–5), coarse identity cues and expression-related geometry are preserved. The dominant residual errors are in high-frequency textures (skin details, hair boundaries, subtle illumination transitions), which is expected for MSE-optimized VAEs under strong compression.

From a training dynamics perspective, reconstruction quality improves sharply in the first 10 epochs and continues to improve more gradually afterward. The final training objective is substantially lower than the initial objective, confirming successful optimization of both reconstruction and KL terms. The model therefore learns a useful low-dimensional manifold, but with the known smoothness-vs-sharpness trade-off of standard VAEs.

B. Generative Performance Evaluation

Experimental results demonstrate effective generative capability: random samples from $z \sim \mathcal{N}(0, I)$ yield coherent facial images with visible diversity in pose, expression intensity, and appearance attributes (Figure 6). While some samples remain slightly blurry, global facial topology is stable and realistic.

Importantly, generated examples do not collapse to a small set of repeated templates. Diversity in generated grids, together with usable interpolation behavior, indicates that the decoder learned a broad region of the latent manifold rather than a narrow memorized subset.

C. Latent Space Structure

The latent space exhibits semantically meaningful structure along three complementary views:

- **Interpolation semantics:** The direction $\delta = \mu_{\text{smile}} - \mu_{\text{non-smile}}$ yields smooth and controllable expression transitions (Figure 7), indicating usable attribute directionality.
- **Class organization:** The t-SNE projection (Figure 8) shows partial smile/non-smile separation with continuity between groups, consistent with shared facial structure and expression overlap.
- **Dimension utilization:** The latent-std profile (Figure 9) indicates non-trivial activity across dimensions rather than complete collapse to a tiny subset.

Together, these observations support the claim that the model learned a structured and navigable representation, even though fully disentangled factors are not guaranteed in this baseline setup.

D. Quantitative Run Summary (120 Epochs)

Table IV consolidates key checkpoints from the canonical script run used to regenerate all report figures.

TABLE IV: Key Metrics from Canonical 120-Epoch Run

Checkpoint	Train Loss	Val Loss
Epoch 1	7226.17	5639.53
Epoch 10	1987.00	1914.21
Epoch 30	1285.54	1470.59
Epoch 60 (best val)	1049.79	1372.88
Epoch 120 (final)	837.18	1406.75

E. Model Selection and Generalization Strategy

The run exhibits a common deep-learning pattern: continuous reduction in training loss with validation improvements peaking earlier (epoch 60). This implies:

- For **best held-out quality**, use the best-validation checkpoint ('best.pt', epoch 60).
- For **full trajectory analysis**, retain the complete 120-epoch curve and final checkpoint ('last.pt', epoch 120).

This separation between optimization progress and generalization peak is important for rigorous reporting and practical deployment.

F. Reproducibility and Artifact Traceability

The full pipeline is script-first and reproducible from the project root:

```
1 source /Users/tahamajs/Documents/uni/venv/bin/
   activate
2 python code/vae_training.py \
3   --data-root train \
4   --out-dir output \
5   --images-dir images \
6   --checkpoint-dir output/checkpoints \
7   --epochs 120 \
8   --batch-size 64 \
9   --lr 5e-4 \
10  --beta 1.0 \
11  --device auto \
12  --num-workers 0 \
13  --analysis-samples 500 \
14  --save-report-plots
```

Primary reproducibility artifacts generated by this run:

- output/train_log.csv: per-epoch train/validation ELBO, reconstruction, and KL.
- output/run_summary.json: normalized metrics summary used for report synchronization.
- output/checkpoints/best.pt and output/checkpoints/last.pt: model selection and final-state checkpoints.
- images/output_cell_*.png: report-bound figures referenced directly in this document.

G. Limitations and Future Work

Current limitations include:

- **Resolution ceiling:** 128×128 inputs limit fine-grained detail fidelity.
- **Late-epoch overfitting:** validation performance degrades after the best checkpoint region.

- **Limited disentanglement control:** baseline ELBO does not explicitly enforce factor disentanglement.
- **Compute constraints:** CPU-only training significantly increases iteration time and limits hyperparameter exploration.

High-value future extensions include:

- β -VAE or KL annealing for improved latent factor structure [3].
- Higher-capacity decoders or perceptual losses to reduce blur.
- Conditional VAE variants for explicit attribute control.
- Stronger model-selection protocol (early stopping on validation trend, multi-seed confidence intervals).

VII. CONCLUSION

This work delivers a complete, reproducible VAE pipeline for facial reconstruction and generation, together with synchronized report artifacts and metrics. The model achieves strong compression (1,536:1), stable generation from the prior, and interpretable latent manipulations. The 120-epoch experimental trace shows substantial optimization progress, with best held-out performance at epoch 60 and expected late-epoch overfitting behavior thereafter.

From an engineering perspective, the project now includes end-to-end reproducibility (script-first execution, checkpointing, figure regeneration, and report synchronization). From a modeling perspective, results validate the utility of variational latent representations for unsupervised facial representation learning, while also highlighting practical opportunities for improved generalization and sharper reconstruction quality in future iterations.

VIII. APPENDIX: FIGURE PROVENANCE

- **Figure: output_cell_11_img_0.png** — Exercise 1 reconstruction sample grid.
- **Figures: output_cell_19_img_0,1,2.png** — Exercise 2 reconstruction grids across batches.
- **Figure: output_cell_21_img_0.png** — Training/validation ELBO and loss curves.
- **Figure: output_cell_23_img_0.png** — Random generation samples from $z \sim \mathcal{N}(0, I)$.
- **Figure: output_cell_25_img_0.png** — Latent interpolation grid between sample pairs.
- **Figure: output_cell_27_img_0.png** — 2D latent visualization (t-SNE) of smile/non-smile clusters.
- **Figure: output_cell_29_img_0.png** — Latent dimension utilization plot.

REFERENCES

- [1] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” *arXiv preprint arXiv:1312.6114*, 2013.
- [2] D. J. Rezende, S. Mohamed, and D. Wierstra, “Stochastic backpropagation and approximate inference in deep generative models,” in *Proc. International Conference on Machine Learning*, 2014, pp. 1278–1286.
- [3] I. Higgins et al., “ β -VAE: Learning basic visual concepts with a constrained variational framework,” in *Proc. ICLR*, 2017.
- [4] A. B. L. Larsen et al., “Autoencoding beyond pixels using a learned similarity metric,” in *Proc. International Conference on Machine Learning*, 2016, pp. 1558–1566.
- [5] K. Sohn, H. Lee, and X. Yan, “Learning structured output representation using deep conditional generative models,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.